Visual Analytics Using Matplotlib and Seaborn

Visual analytics is the process of exploring, analyzing, and interpreting data using charts, graphs, and visual representations to discover patterns, trends, relationships, and unusual observations

Feature	Matplotlib	Seaborn
Main purpose	General-purpose plotting	Statistical data visualization
DataFrame support	Supported	Excellent built-in support
Customization	Very extensive	Convenient, with Matplotlib customization available
Statistical plots	Possible, but often requires more code	Built-in functions for many statistical plots
Default appearance	Basic	Generally more polished
Best suited for	Fine-grained control and custom charts	Quick exploration of statistical patterns

Frequently used plotting functions

Function	Purpose
plt.plot()	Line chart
plt.bar()	Vertical bar chart
plt.barh()	Horizontal bar chart
plt.scatter()	Scatter plot
plt.hist()	Histogram
plt.pie()	Pie chart
plt.subplots()	Creates a figure and one or more axes
plt.savefig()	Saves a figure to a file
plt.show()	Displays a figure

Line chart using Matplotlib

A line chart is useful for showing how a numerical value changes over an ordered sequence, particularly time

```
sales_df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 13500, 18000, 22000, 25000]
})
plt.figure(figsize=(8, 4))
plt.plot(
    sales_df["Month"],
    sales_df["Sales"],
    marker="o",
    linestyle="-",
    linewidth=2,
    label="Sales"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales (₹)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

plt.figure(figsize=(8, 4)) is used to create a new figure (plotting area) with a specified size

plt.figure() creates a new figure where the graph will be drawn

figsize=(8, 4) specifies the figure's dimensions in inches

8 = width of the figure in inches

4 = height of the figure in inches

Parameter	Explanation
sales_df["Month"]	X-axis values (months)
sales_df["Sales"]	Y-axis values (sales figures)
marker="o"	Displays a circular marker at each data point
linestyle="-"	Connects the data points using a solid line
linewidth=2	Sets the line thickness to 2 points
label="Sales"	Assigns the name "Sales" to the line for use in a legend

label="Sales" assigns the name Sales to the plotted line

plt.legend() displays that name in a legend box on the graph

plt.show() displays the graph

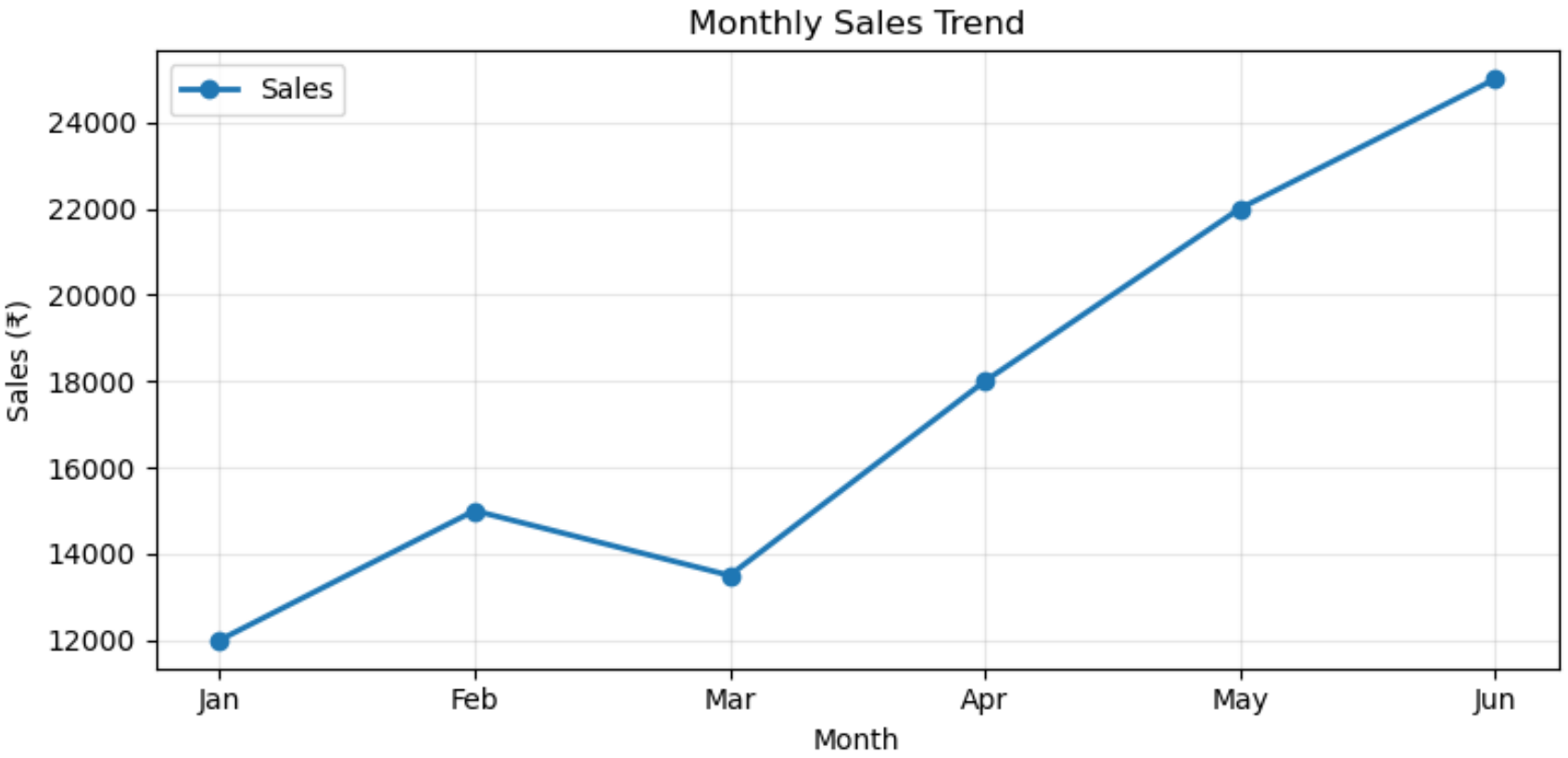
plt.grid() - Controls the display of grid lines

True - Turns the grid lines on

alpha=0.3 Sets the grid-line opacity to 30%, making the lines appear light

plt.tight_layout()

This statement automatically adjusts the spacing around graph elements to prevent labels, titles, and axis names from overlapping or being cut off

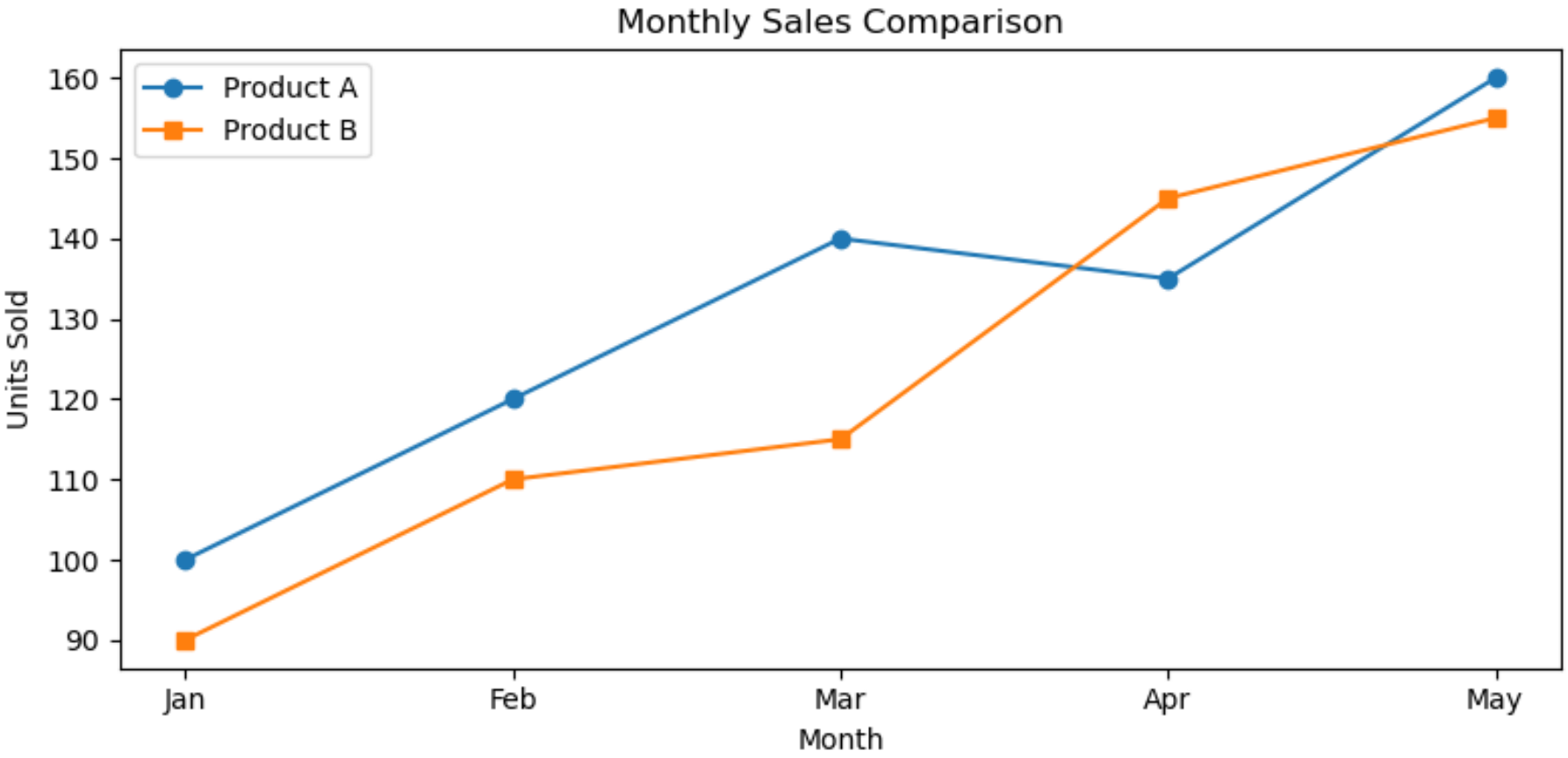

Monthly Sales Trend
Sales
24000
22000
20000
18000
16000
14000
12000
Sales (₹)
Jan
Feb
Mar
Apr
May
Jun
Month

Compare two products

```
sales_df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Product_A": [100, 120, 140, 135, 160],
    "Product_B": [90, 110, 115, 145, 155]
})
plt.figure(figsize=(8, 4))

plt.plot(sales_df["Month"], sales_df["Product_A"],
         marker="o", label="Product A")
plt.plot(sales_df["Month"], sales_df["Product_B"],
         marker="s", label="Product B")

plt.title("Monthly Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Units Sold")
plt.legend()
plt.tight_layout()
plt.show()
```


Monthly Sales Comparison
Product A
Product B
Units Sold
160
150
140
130
120
110
100
90
Jan
Feb
Mar
Apr
May
Month

```
import matplotlib.pyplot as plt

students = ["Asha", "Rahul", "John", "Priya"]
marks = [65, 80, 45, 90]

plt.plot(students, marks, marker="o")

plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Student Marks")

plt.ylim(0, 100)

plt.show()
```

Code	Effect
plt.ylim(0, 100)	Y-axis ranges from 0 to 100
plt.ylim(0, 50)	Y-axis ranges from 0 to 50
plt.ylim(20, 100)	Y-axis ranges from 20 to 100
plt.ylim(-10, 100)	Y-axis ranges from −10 to 100

What happens if we change the values?

plt.ylim() changes the visible Y-axis range, not the actual data values

If a data point falls outside the specified limits, it will not be visible within the plotting area

Related function: plt.xlim(0, 10) sets the minimum and maximum values of the X-axis.

Bar chart using Matplotlib

A bar chart compares values across categories

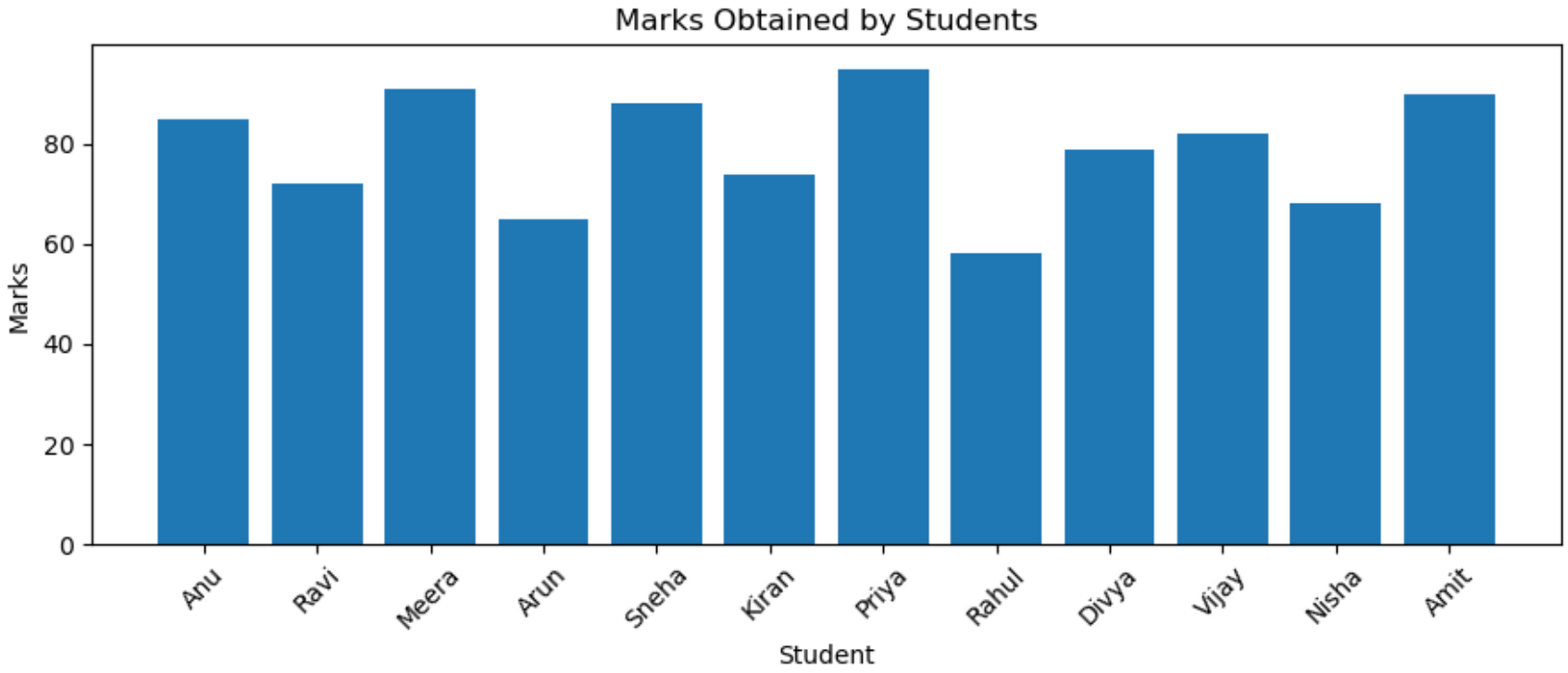
Creating a DataFrame for practical examples

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
data = {
    "Student": [
        "Anu", "Ravi", "Meera", "Arun", "Sneha",
        "Kiran", "Priya", "Rahul", "Divya", "Vijay",
        "Nisha", "Amit"
    ],
    "Department": [
        "CSE", "CSE", "ECE", "ECE", "CSE", "ECE",
        "CSE", "ECE", "CSE", "ECE", "CSE", "ECE"
    ],
    "Marks": [85, 72, 91, 65, 88, 74, 95, 58, 79, 82, 68, 90],
    "Attendance": [92, 78, 95, 68, 90, 80, 98, 60, 85, 88, 72, 94],
    "Study_Hours": [5, 3, 6, 2, 5, 3, 7, 2, 4, 5, 3, 6],
    "Assignment_Score": [90, 75, 94, 68, 89, 77, 96, 60, 82, 85, 70, 92]
}
df = pd.DataFrame(data)
print(df)
```

Inspect the dataset before visualization

```
print(df.head())
print(df.shape)
print(df.columns)
print(df.dtypes)
print(df.describe())
```

```
plt.figure(figsize=(9, 4))
plt.bar(df["Student"], df["Marks"])
plt.title("Marks Obtained by Students")
plt.xlabel("Student")
plt.ylabel("Marks")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```


Marks Obtained by Students
Marks
0
20
40
60
80
Anu
Ravi
Meera
Arun
Sneha
Kiran
Priya
Rahul
Divya
Vijay
Nisha
Amit
Student

plt.xticks() controls the tick marks and labels on the X-axis

rotation=45 rotates the X-axis labels by 45 degrees

This is especially useful when displaying month names, product names, dates, or category labels

when plotting sales for January, February, September, and other long category names, horizontal labels may overlap

Rotating them by 45 degrees can improve readability

Average marks by department

```
dept_avg = df.groupby("Department")["Marks"].mean()

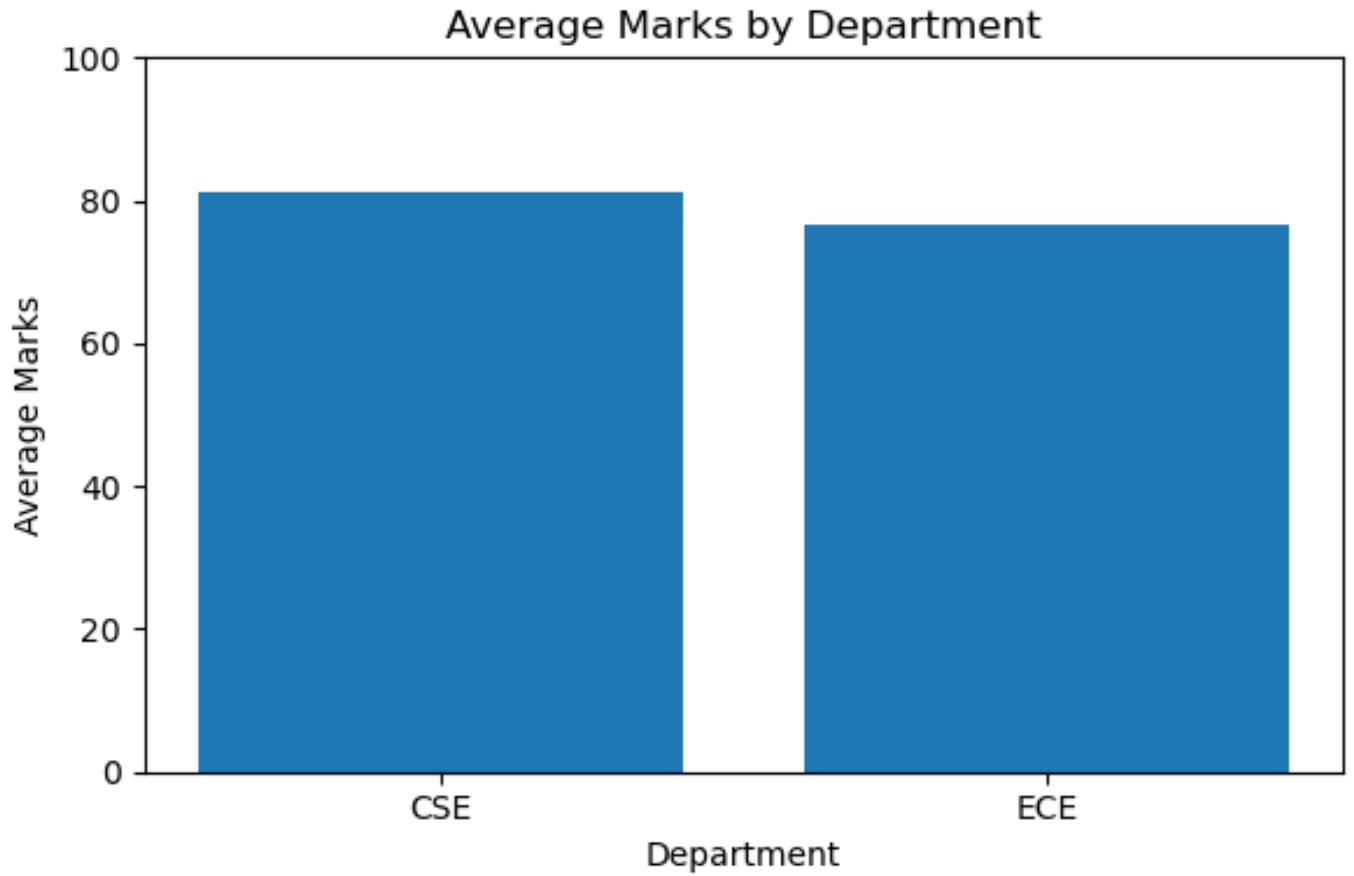
print(dept_avg)

plt.figure(figsize=(6, 4))

plt.bar(dept_avg.index, dept_avg.values)

plt.title("Average Marks by Department")
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
```

plt.ylim() controls the limits of the Y-axis.
0 specifies the minimum Y-axis value.
100 specifies the maximum Y-axis value.
Therefore, the Y-axis displays values from 0 to 100.


Average Marks by Department
100
80
60
40
20
0
Average Marks
CSE
ECE
Department

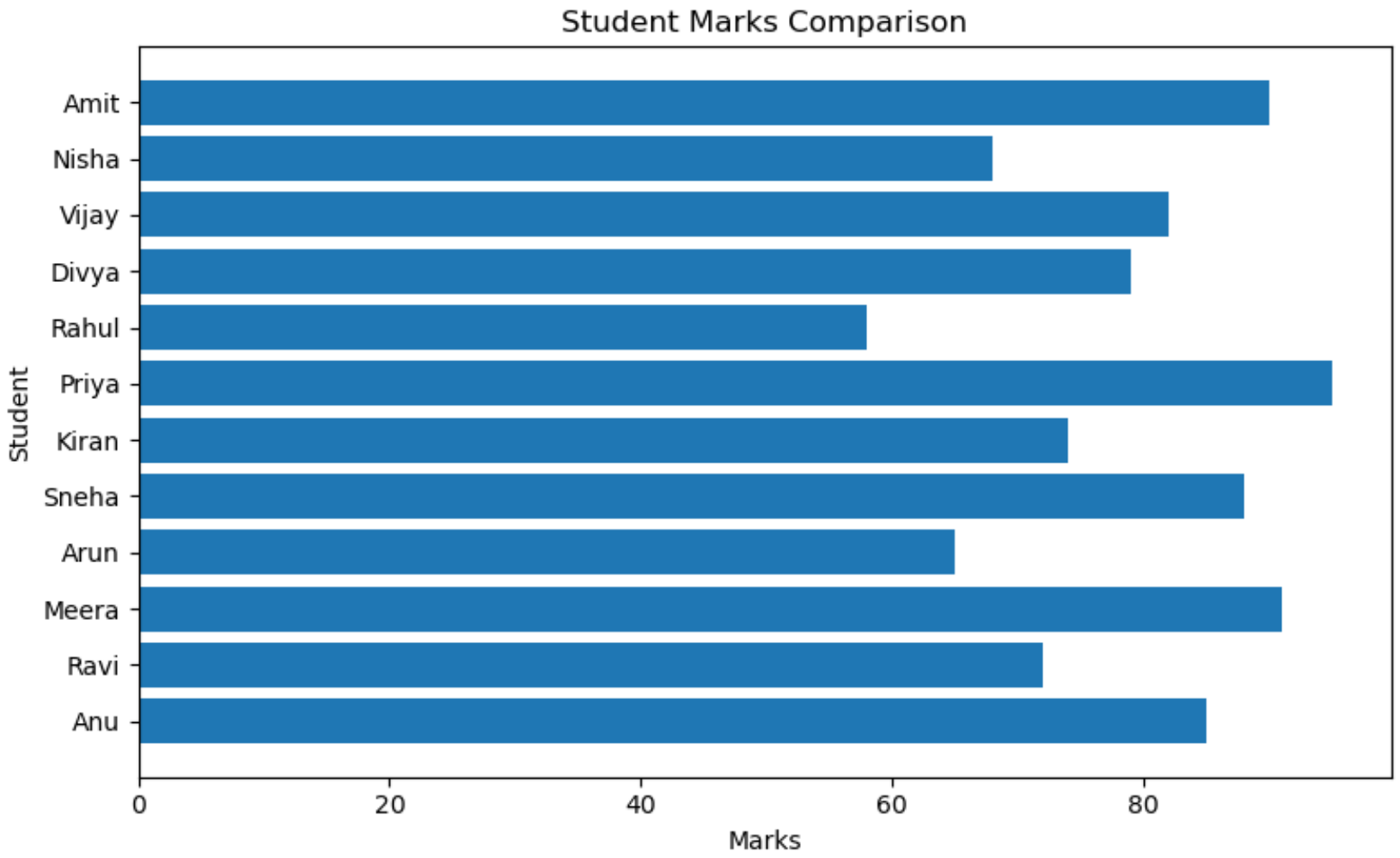
Horizontal bar chart

- Horizontal bars are useful when category labels are long.

```
plt.figure(figsize=(8, 5))

plt.barh(df["Student"], df["Marks"])

plt.title("Student Marks Comparison")
plt.xlabel("Marks")
plt.ylabel("Student")
plt.tight_layout()
plt.show()
```


Student Marks Comparison
Amit
Nisha
Vijay
Divya
Rahul
Priya
Kiran
Sneha
Arun
Meera
Ravi
Anu
Student
0
20
40
60
80
Marks

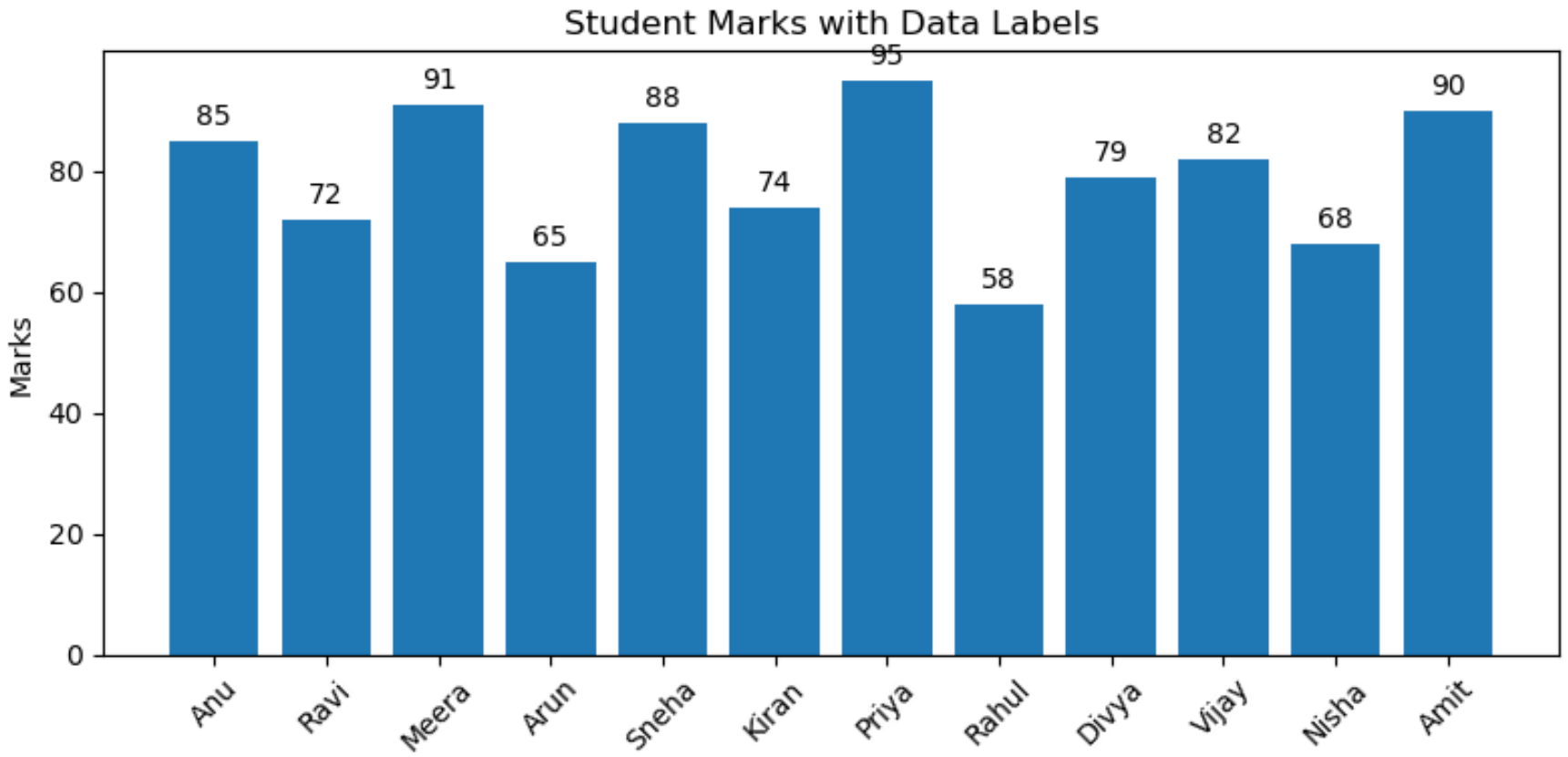
Add values above the bars

```
fig, ax = plt.subplots(figsize=(8, 4))

bars = ax.bar(df["Student"], df["Marks"])

ax.bar_label(bars, fmt="%.0f", padding=3)
ax.set_title("Student Marks with Data Labels")
ax.set_ylabel("Marks")
ax.tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```


Student Marks with Data Labels
Marks
0
20
40
60
80
85
72
91
65
88
74
95
58
79
82
68
90
Anu
Ravi
Meera
Arun
Sneha
Kiran
Priya
Rahul
Divya
Vijay
Nisha
Amit

fig, ax = plt.subplots(figsize=(8, 4))

This statement creates the figure and the plotting area

plt.subplots() creates a figure and one or more axes

fig refers to the complete figure

ax refers to the axes containing the graph

figsize=(8, 4) sets the figure width to 8 inches and height to 4 inches

bars = ax.bar(df["Student"], df["Marks"])

This statement creates a vertical bar chart

ax.bar() draws vertical bars

df["Student"] provides the X-axis categories

df["Marks"] provides the height of each bar

bars stores the collection of bar objects returned by Matplotlib.

ax.bar_label(bars, fmt="%.0f", padding=3)

This statement displays the numerical value of each bar above it.

Parameter	Explanation
bars	The bar objects created by ax.bar()
fmt="%.0f"	Displays values as whole numbers without decimal places
padding=3	Adds 3 points of space between each bar's top and its label

For example, the value 75 will appear above Asha's bar

What does fmt="%.0f" mean?

%f formats a number as a floating-point value.

.0 specifies zero digits after the decimal point.

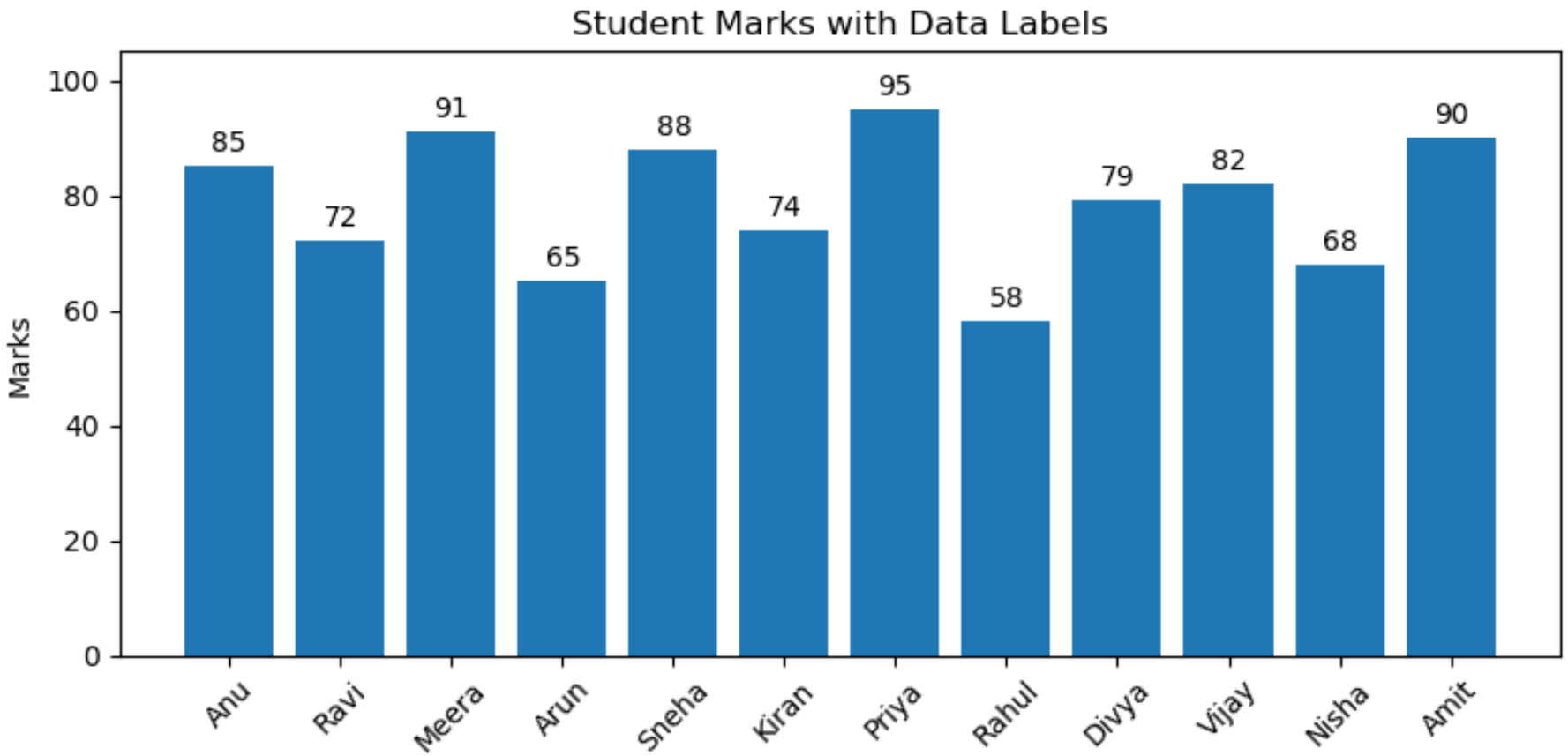
Therefore, 75.0 is displayed as 75.

If you use fmt="%.2f", a value such as 75 will be displayed as 75.00.

- Suppose the highest student's mark is 98, and the data label 98 appears outside the top border

- Use ax.set_ylim() to provide extra space above the tallest bar

```
ax.set_ylim(0, df["Marks"].max() + 10)
plt.tight_layout()
plt.show()
```


Student Marks with Data Labels
Marks
0
20
40
60
80
100
85
72
91
65
88
74
95
58
79
82
68
90
Anu
Ravi
Meera
Arun
Sneha
Kiran
Priya
Rahul
Divya
Vijay
Nisha
Amit

Histogram: Understanding data distribution

- A histogram groups numerical values into intervals called bins and displays how many observations fall within each interval

- Unlike a bar chart, a histogram is primarily used for numerical distributions, not comparisons between unrelated categories

```
plt.figure(figsize=(7, 4))

plt.hist(df["Marks"], bins=6, edgecolor="black")

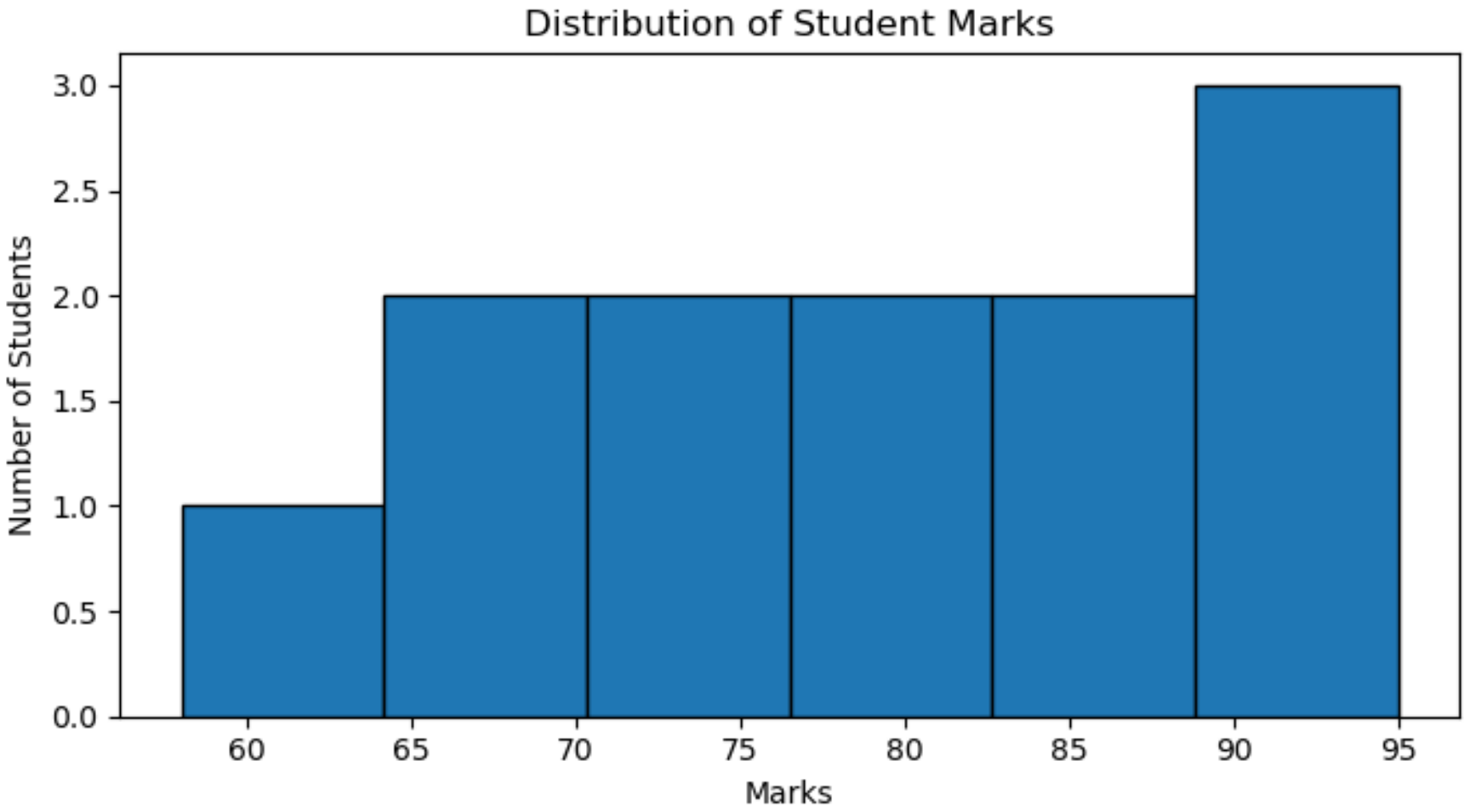
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.tight_layout()
plt.show()
```

bins=6: divides the numerical range into six intervals

edgecolor="black": outlines the bars

The vertical axis displays the number of observations in each interval

Interpretation: The histogram shows where student marks are concentrated and whether the distribution appears symmetric, skewed, or spread out.


Distribution of Student Marks
Number of Students
Marks
0.0
0.5
1.0
1.5
2.0
2.5
3.0
60
65
70
75
80
85
90
95

Change the number of bins

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

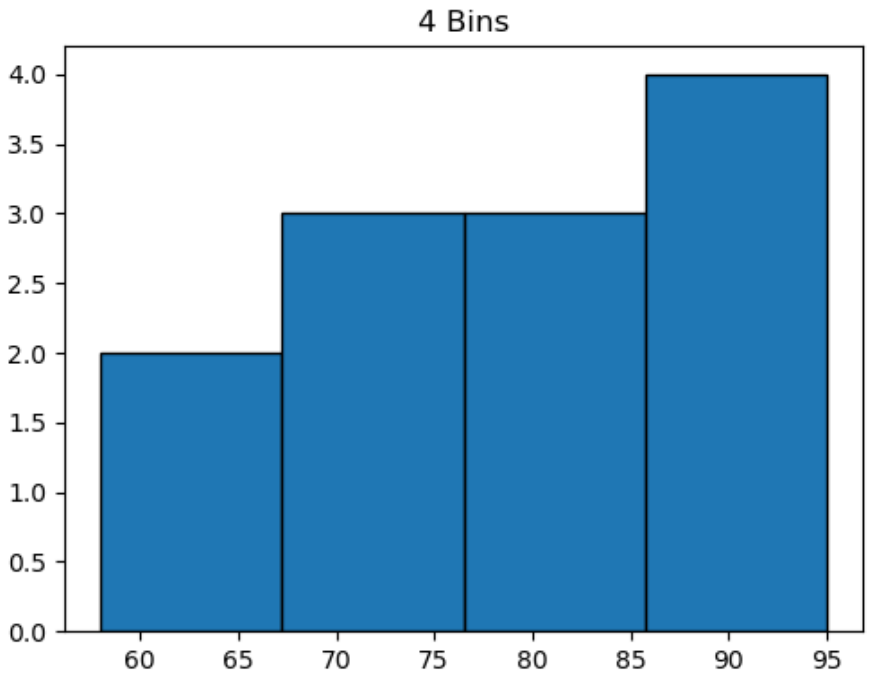
axes[0].hist(df["Marks"], bins=4, edgecolor="black")
axes[0].set_title("4 Bins")

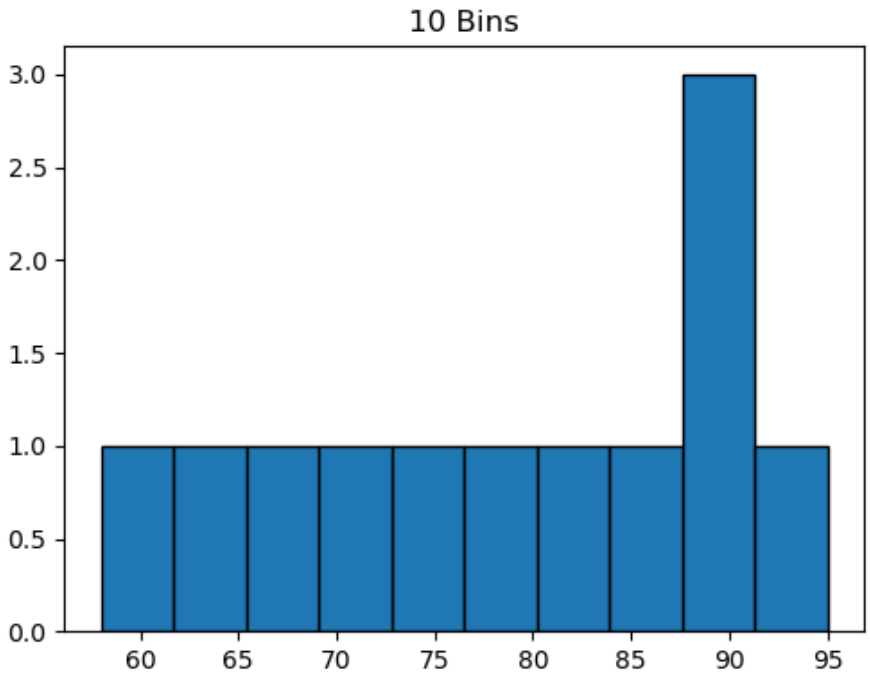
axes[1].hist(df["Marks"], bins=10, edgecolor="black")
axes[1].set_title("10 Bins")

plt.tight_layout()
plt.show()
```

fig, axes = plt.subplots(1, 2, figsize=(10, 4)) creates two graphs side by side within a single figure

Code	Explanation
plt.subplots()	Creates a figure and one or more plotting areas.
1	Specifies 1 row of graphs.
2	Specifies 2 columns of graphs.
figsize=(10, 4)	Sets the total figure width to 10 inches and height to 4 inches.
fig	Stores the complete figure.
axes	Stores the two Axes objects, one for each graph.

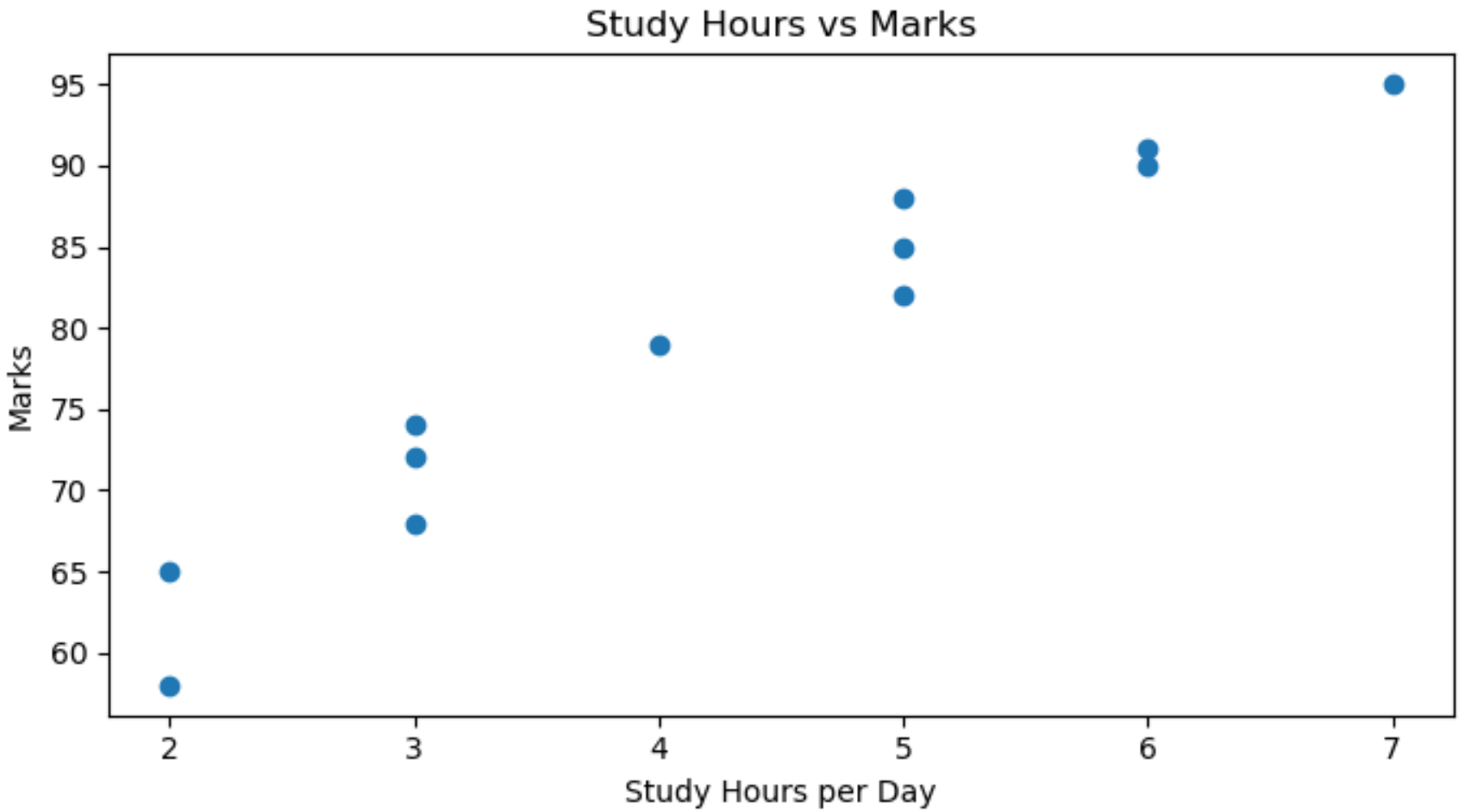

4 Bins
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
60
65
70
75
80
85
90
95


10 Bins
3.0
2.5
2.0
1.5
1.0
0.5
0.0
60
65
70
75
80
85
90
95

Scatter plot: Studying relationships

Study hours versus marks

```
plt.figure(figsize=(7, 4))
plt.scatter(df["Study_Hours"], df["Marks"])
plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours per Day")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```


Study Hours vs Marks
Marks
95
90
85
80
75
70
65
60
2
3
4
5
6
7
Study Hours per Day

A scatter plot can reveal:

- Positive relationships
- Negative relationships
- Weak or absent relationships
- Clusters of observations
- Unusual data points

Use point size and transparency

```
plt.figure(figsize=(7, 4))

plt.scatter(
    df["Study_Hours"],
    df["Marks"],
    s=100,
    alpha=0.7
)

plt.title("Study Hours and Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

s=100 controls marker size.
alpha=0.7 makes markers partially transparent, which can help when observations overlap.

Pie chart

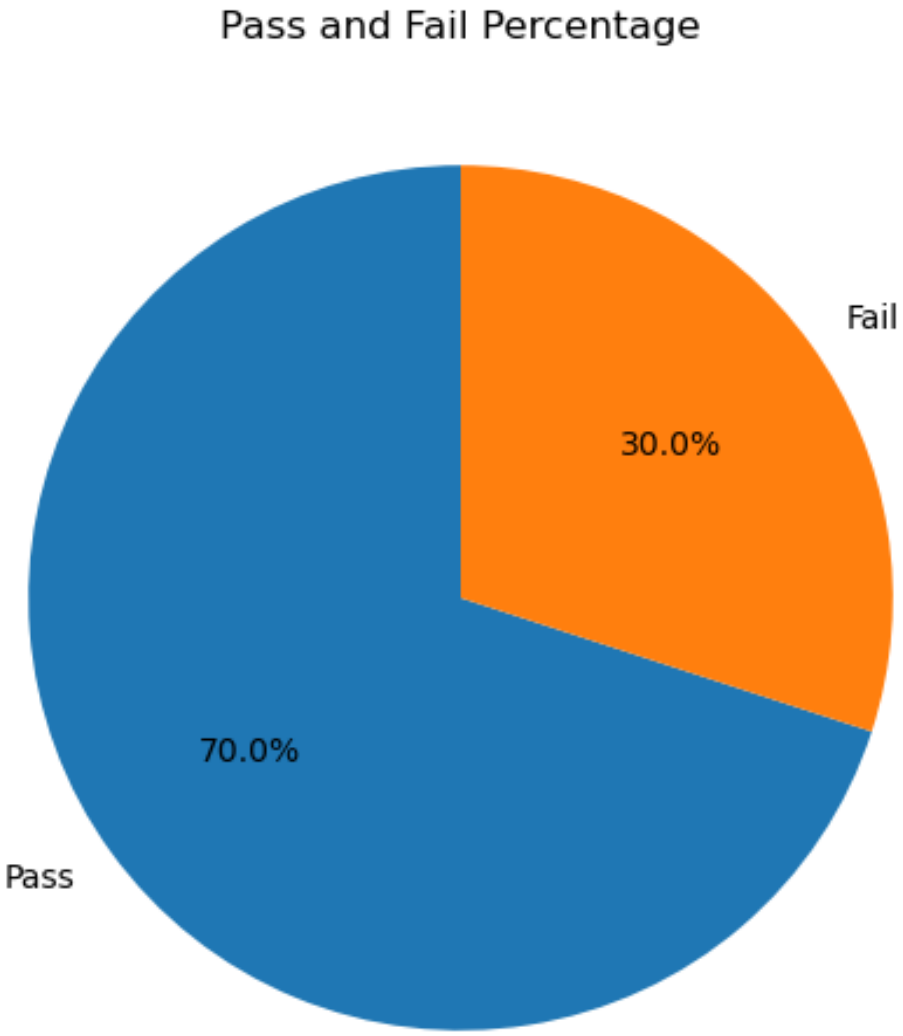
- A pie chart shows how a total is divided among categories
- It is most suitable when there are a small number of categories.

```
results = pd.Series([
    "Pass", "Pass", "Fail", "Pass", "Fail",
    "Pass", "Pass", "Fail", "Pass", "Pass"
])

counts = results.value_counts()

plt.figure(figsize=(6, 6))
plt.pie(
    counts.values,
    labels=counts.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Pass and Fail Percentage")
plt.show()
```

Parameter	Explanation
counts.values	Provides the numerical values that determine the size of each pie slice.
labels=counts.index	Displays the category names beside the corresponding slices.
autopct="%1.1f%%"	Displays the percentage on each slice, rounded to one decimal place.
startangle=90	Rotates the pie chart so that the first slice starts at 90 degrees.


Pass and Fail Percentage
Fail
30.0%
70.0%
Pass

Data visualization directly from Pandas DataFrames

- Pandas provides a convenient .plot() interface that works with Matplotlib
- It is useful when data is already organized in a DataFrame.

DataFrame line plot

```
monthly_df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [100, 130, 125, 160, 190],
    "Expenses": [70, 85, 90, 100, 120]
})
monthly_df.plot(
    x="Month",
    y=["Sales", "Expenses"],
    kind="line",
    marker="o",
    figsize=(8, 4)
)
plt.title("Monthly Sales and Expenses")
plt.ylabel("Amount")
plt.tight_layout()
plt.show()
```

DataFrame bar plot

```
dept_df = df.groupby("Department")["Marks"].mean()

dept_df.plot(kind="bar", figsize=(6, 4))

plt.title("Department-wise Average Marks")
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

DataFrame histogram

```
df["Marks"].plot(
    kind="hist",
    bins=6,
    edgecolor="black",
    figsize=(7, 4)
)

plt.title("Distribution of Marks")
plt.xlabel("Marks")
plt.show()
```

Statistical visualization using Seaborn

sns.set_theme(style="whitegrid")

Set a visual theme

Common styles include:

"whitegrid" — white background with grid lines.

"white" — plain white background.

"darkgrid" — darker background with grid lines.

"ticks" — emphasizes axis tick marks.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Student": [
        "Anu", "Ravi", "Meera", "Arun", "Sneha",
        "Kiran", "Priya", "Rahul", "Divya", "Vijay",
        "Nisha", "Amit"
    ],
    "Department": [
        "CSE", "CSE", "ECE", "ECE", "CSE", "ECE",
        "CSE", "ECE", "CSE", "ECE", "CSE", "ECE"
    ],
    "Marks": [85, 72, 91, 65, 88, 74, 95, 58, 79, 82, 68, 90],
    "Attendance": [92, 78, 95, 68, 90, 80, 98, 60, 85, 88, 72, 94],
    "Study_Hours": [5, 3, 6, 2, 5, 3, 7, 2, 4, 5, 3, 6],
    "Assignment_Score": [90, 75, 94, 68, 89, 77, 96, 60, 82, 85, 70, 92]
}
df = pd.DataFrame(data)
print(df)
```

Seaborn bar plot

- A Seaborn barplot() generally calculates an aggregate statistic, such as the mean, for each category

- It is not the same as Matplotlib's bar(), which directly plots the supplied heights

Average marks by department

```
plt.figure(figsize=(7, 4))

sns.barplot(
    data=df,
    x="Department",
    y="Marks"
)

plt.title("Average Marks by Department")
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.show()
```

Compare departments by gender

```
gender_df = pd.DataFrame({
    "Student": ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L"],
    "Department": ["CSE", "CSE", "CSE", "CSE", "ECE", "ECE",
                   "ECE", "ECE", "CSE", "ECE", "CSE", "ECE"],
    "Gender": ["Female", "Male", "Female", "Male", "Female",
"Male", "Female", "Male", "Female", "Male", "Male", "Female"],
    "Marks": [85, 72, 95, 79, 91, 65, 74, 58, 88, 82, 68, 90]
})
```

Plot the average marks for each gender within each department.

```
plt.figure(figsize=(8, 4))
sns.barplot(
    data=gender_df,
    x="Department",
    y="Marks",
    hue="Gender"
)
plt.title("Average Marks by Department and Gender")
plt.show()
```

The hue parameter divides each department's bars by gender

Seaborn count plot

A count plot displays the number of observations in each category.

Number of students in each department

```
plt.figure(figsize=(6, 4))
sns.countplot(
    data=df,
    x="Department"
)
plt.title("Number of Students by Department")
plt.xlabel("Department")
plt.ylabel("Student Count")
plt.show()
```

Seaborn histogram

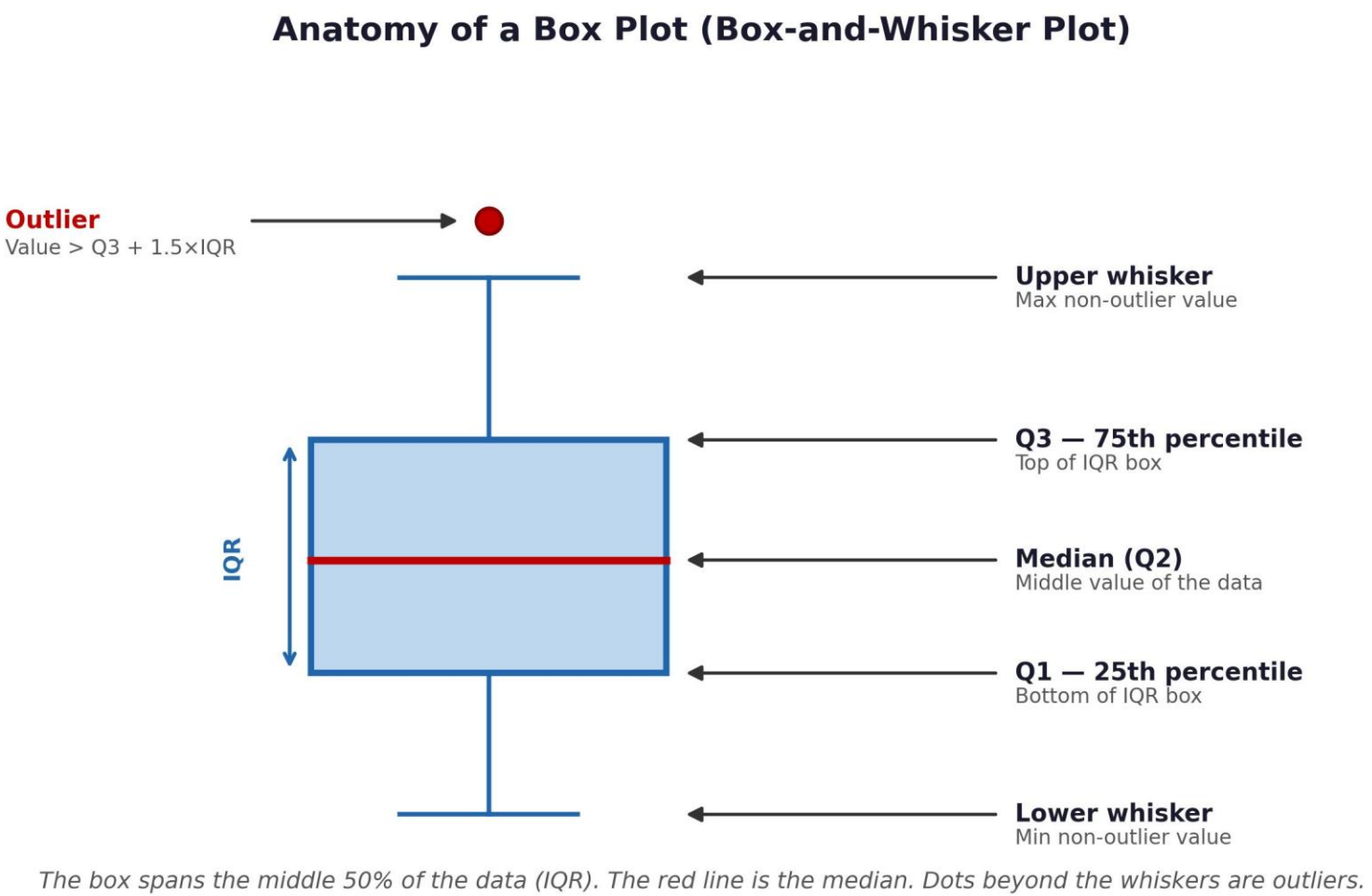
```
plt.figure(figsize=(7, 4))

sns.histplot(
    data=df,
    x="Marks",
    bins=6,
    kde=True
)

plt.title("Distribution of Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

data=df: use the student DataFrame.
x="Marks": select the marks column.
bins=6: use six histogram intervals.
kde=True: overlay a smoothed estimate of the distribution.

Box plot: Detecting spread and potential outliers



```
plt.figure(figsize=(7, 4))

sns.boxplot(
    data=df,
    x="Department",
    y="Marks"
)

plt.title("Marks Distribution by Department")
plt.ylabel("Marks")
plt.show()
```

Scatter plot using Seaborn

Attendance versus marks

```
plt.figure(figsize=(7, 4))

sns.scatterplot(
    data=df,
    x="Attendance",
    y="Marks"
)

plt.title("Attendance vs Marks")
plt.xlabel("Attendance (%)")
plt.ylabel("Marks")
plt.show()
```

Add department information using color

```
plt.figure(figsize=(7, 4))

sns.scatterplot(
    data=df,
    x="Attendance",
    y="Marks",
    hue="Department",
    s=100
)

plt.title("Attendance and Marks by Department")
plt.show()
```

hue="Department"	Assigns different colors to different departments.
s=100	Sets the marker size to 100 points squared.

Add study hours using point size

```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="Attendance",
    y="Marks",
    hue="Department",
    size="Study_Hours",
    sizes=(50, 250)
)

plt.title("Student Performance: Attendance, Marks and Study Hours")
plt.show()
```

Regression plot

A regression plot displays observations and fits a statistical trend line

```
plt.figure(figsize=(7, 4))

sns.regplot(
    data=df,
    x="Study_Hours",
    y="Marks"
)

plt.title("Relationship Between Study Hours and Marks")
plt.show()
```

Correlation coefficient

Calculate the Pearson correlation between study hours and marks.

```
correlation = df["Study_Hours"].corr(df["Marks"])

print("Correlation:", round(correlation, 3))
```

- The Pearson correlation coefficient ranges from −1 to +1:
- Near +1: strong positive linear association.
- Near -1: strong negative linear association.
- Near 0: weak or no linear association.

Heatmap: Correlation analysis

- A heatmap represents numerical values using colors
- One common application is displaying a correlation matrix

```
numeric_df = df[
    ["Marks", "Attendance", "Study_Hours",
"Assignment_Score"]
]

corr_matrix = numeric_df.corr()

print(corr_matrix)
```

- The correlation matrix contains the pairwise Pearson correlations between the numerical variables

- Now visualize it

```
plt.figure(figsize=(8, 6))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

annot=True: writes the correlation values inside the cells.
cmap="coolwarm": uses a diverging color palette.
vmin=-1, vmax=1: fixes the scale to the correlation range.
fmt=".2f": displays values to two decimal places.

Pair plot: Visualizing multiple relationships

- A pair plot creates a collection of plots for pairs of numerical variables
- The diagonal often displays each variable's distribution.

```
sns.pairplot(
    df,
    vars=[
        "Marks",
        "Attendance",
        "Study_Hours",
        "Assignment_Score"
    ],
    diag_kind="hist"
)

plt.show()
```

Color the pair plot by department

```
sns.pairplot(
    df,
    vars=["Marks", "Attendance", "Study_Hours"],
    hue="Department",
    diag_kind="hist"
)

plt.show()
```

Real-world practical example: Sales data analysis

```
sales = pd.DataFrame({
    "Region": [
        "South", "North", "South", "East", "North",
        "East", "South", "North", "East", "South"
    ],
    "Product": [
        "Laptop", "Mobile", "Mobile", "Laptop", "Laptop",
        "Mobile", "Laptop", "Mobile", "Laptop", "Mobile"
    ],
    "Sales": [
        50000, 30000, 25000, 45000, 55000,
        35000, 60000, 32000, 40000, 28000
    ],
    "Units_Sold": [5, 10, 8, 4, 6, 12, 7, 9, 4, 7],
    "Profit": [8000, 5000, 4000, 7000, 9000,
               6000, 10000, 4500, 6500, 4200]
})

print(sales)
```

- Question 1: Which region generates the highest sales?
- Question 2: Which product generates more profit?
- Question 3: Is sales revenue related to profit?
- Question 4: What is the correlation between sales and profit?

Saving charts to files

Charts can be saved for reports, presentations, and assignments.

```
plt.figure(figsize=(8, 4))

plt.bar(dept_avg.index, dept_avg.values)

plt.title("Department-wise Average Marks")
plt.xlabel("Department")
plt.ylabel("Average Marks")

plt.tight_layout()

plt.savefig(
    "department_marks.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

dpi=300: produces a high-resolution raster image.
bbox_inches="tight": trims excess whitespace.
"department_marks.png": output filename.

Save a Seaborn chart

```
fig, ax = plt.subplots(figsize=(8, 4))

sns.boxplot(
    data=df,
    x="Department",
    y="Marks",
    ax=ax
)

ax.set_title("Marks Distribution by Department")

fig.tight_layout()
fig.savefig("department_boxplot.png", dpi=300)
plt.show()
```

Thank You..